

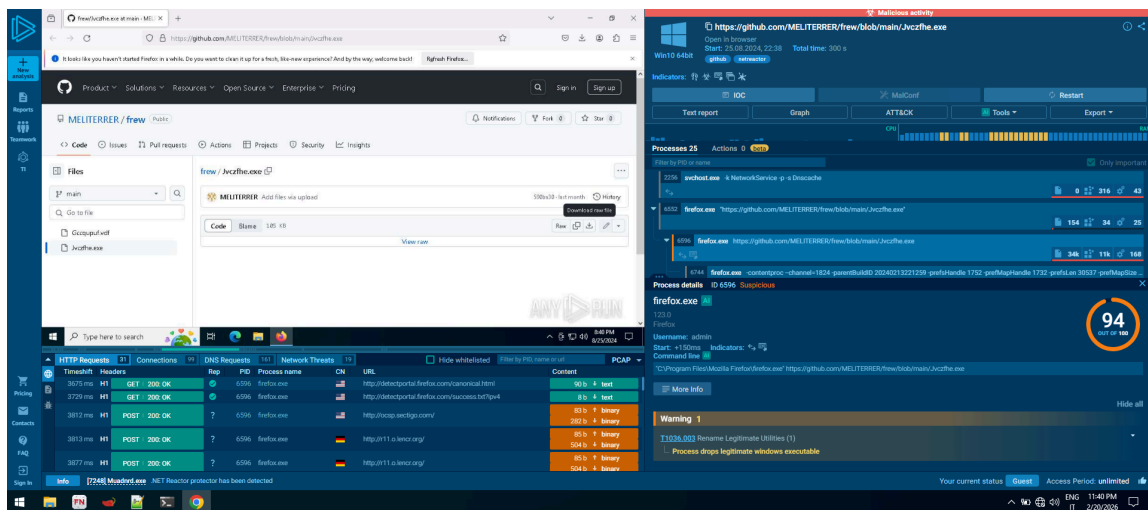
Task 2: Analisi Forense e Tattica di un Campione Malware

Data: 20/02/2026

Analista: Daniele di Martino - Bkm4ge

-----Introduzione e Metodologia di Analisi

Il presente documento costituisce un **Report di Analisi Comportamentale e Forense** di un campione di codice malevolo, condotta attraverso l'ambiente di sandboxing dinamico **ANY.RUN**. L'obiettivo primario è la dissezioni della catena della Kill Chain, l'identificazione delle tecniche di evasione a livello di rete e di *runtime*, e l'inferenza sul potenziale obiettivo del payload.



Riferimento Analisi Completa:

<https://app.any.run/tasks/9a15871843fe-45ce-85b366203dbc2281/>-----1. Analisi Tattica e Vettore di Infezione (Initial Access & Defense Evasion)1.1. Vettore di Accesso Iniziale e Evasione Perimetrale (*Living off the Trusted Sites - LOTS*)

Il vettore di infezione sfrutta una tecnica avanzata di **Evasione delle Difese Perimetrali** nota come *Living off the Trusted Sites (LOTS)*.

Nel riquadro di sinistra è visibile il browser Firefox navigare sugli URL

[https://github.com/MELITERRER/kioluu/...](https://github.com/MELITERRER/kioluu/) e

[https://github.com/MELITERRER/frew/...](https://github.com/MELITERRER/frew/) Nel riquadro di destra nell'albero dei processi, gli argomenti della riga di comando di **firefox.exe** confermano gli stessi URL

L'attore della minaccia ha architettato l'attacco ospitando i payload binari primari (**Jvczfhe.exe** e **muadnrd.exe**) su repository pubbliche e apparentemente innocui sulla piattaforma **GitHub** (associati all'account MELITERRER, progetti frew e kioluu).

Implicazioni di Sicurezza:

Il traffico di download dei payload avviene su protocollo **HTTPS** e coinvolge domini con alta reputazione come raw.githubusercontent.com. Questa scelta tattica permette di:

1. **Bypassare i Firewall di Rete (NGFW):** I filtri basati sulla reputazione dei domini e sulla categorizzazione web classificano GitHub come una risorsa legittima, consentendo l'accesso ai payload.
2. **Mascherare il Traffico C2:** Le comunicazioni successive potrebbero essere camuffate tra il normale traffico web cifrato.

1.2. Offuscamento e Protezione del Codice a Riposo (*Packer*)

L'analisi statica del campione binario è stata deliberatamente resa inefficace attraverso l'uso di tecniche di offuscamento commerciale, mirate a prevenire il *Reverse Engineering* e l'analisi automatizzata da parte degli *scanner* antivirus (AV):

- **Identificazione del Packer:** I metadati e i log di base indicano l'uso di **.NET Reactor protector**. Questo è un *packer* commerciale noto per la sua robustezza nel cifrare e oscurare l'intera logica di un eseguibile basato su .NET Framework.
- **Impedimento all'Analisi Statica:** La crittografia del codice a riposo rende impossibile per gli strumenti di *decompiler* o gli *scanner* AV leggere le stringhe, le API chiamate o l'intento malevolo del file su disco.
- **Effimericità del Payload:** L'assenza dell'impronta SHA256 dei binari su database di minacce noti (es. MalwareBazaar) suggerisce che i payload sono **altamente effimeri** o generati dinamicamente (polimorfici) per ogni vittima, riducendo drasticamente l'efficacia delle difese basate sulle firme.

Conclusione Tattica sul Codice:

Il malware non è stato concepito per essere rilevato dal file system. Il codice malevolo diventa visibile e operativo solo al momento dell'esecuzione, quando viene **decriptato e iniettato direttamente nella memoria RAM** di processi legittimi (**installutil.exe** e **muadnrd.exe** stesso)

.-----2. La Catena di Esecuzione Biforcuta (Kill Chain e Evasion at Runtime)

L'analisi dinamica rivela una sofisticata catena di esecuzione a due rami paralleli, progettata per la ridondanza e il *bypass* selettivo delle difese EDR.-----Matrice della Catena di Infezione



Processo	Ruolo nella Catena di Infezione	Tattiche/ID MITRE ATT&CK (Esempi)	Dettagli Operativi Chiave
firefox.exe	Accesso Iniziale	T1190 (Valid Accounts/Software)	Vettore iniziale del download dei due stager.
Jvczfhe.exe	Stager Primario / Decryptor	T1027.004 (.NET Reactor)	Primo binario offuscato eseguito, genera due rami paralleli.
cmd.exe -> timeout.exe	Evasione (Anti-Sandbox)	T1497.003 (Sleep/Delay)	Introduce un ritardo forzato per esaurire il

			tempo limite di monitoraggio della sandbox.
installutil.exe	Esecuzione (LOLBin)	T1218 (Process Hollowing)	Binario Microsoft legittimo (.NET Framework) usato per ospitare il payload iniettato.
muadnrd.exe	Stager Secondario / Failsafe	T1055 (Process Injection)	Secondo payload indipendente per ridondanza.
muadnrd.exe (Processo Figlio)	Esecuzione (Self-Injection)	T1055.001 (RunPE/Self-Injection)	Inietta il codice in un'istanza di sé stesso, isolando l'esecuzione.
werfault.exe	Diagnostica	T1497 (Discovery/Evasion)	Attivazione del Windows Error Reporting, spesso risultato di crash controllati o inattesi durante l'esecuzione evasiva.

2.1. Ramo Primario: Esecuzione tramite LOLBin (*Process Hollowing*)

Il primo percorso d'attacco si concentra sull'abuso di un binario di sistema legittimo (LOLBin - *Living Off the Land Binary*):

1. **Avvio Stager e Delay:** **Jvczfhe.exe** esegue lo sleep tramite **timeout.exe**.
2. **Abuso di LOLBin:** **Jvczfhe.exe** avvia una nuova istanza di **installutil.exe** (utility di installazione .NET).
3. **Process Hollowing:** Il codice malevolo decriptato viene iniettato nello spazio di memoria di **installutil.exe**, sostituendo il codice originale.
4. **Obiettivo:** Le attività malevole (es. connessioni C2 o alterazioni del sistema) vengono attribuite dal sistema operativo a un processo firmato e attendibile di Microsoft, eludendo la maggior parte delle euristiche EDR basate sul monitoraggio della reputazione.

2.2. Ramo Secondario: Esecuzione Redundante (*Self-Injection*)

Il secondo percorso opera come un meccanismo di *failsafe* o un veicolo per un modulo con diverse funzionalità:

1. **Avvio Stager e Delay:** `muadnrd.exe` replica l'evasione temporale tramite `timeout.exe`.
2. **Self-Injection (RunPE):** `muadnrd.exe` crea un processo figlio omonimo e inietta il codice malevolo decriptato al suo interno.
3. **Obiettivo:** Questo metodo assicura che il payload venga eseguito anche in ambienti in cui l'iniezione in LOLBin (Vettore 1) è fallita, offrendo una forte **ridondanza**. Inoltre, l'isolamento del codice attivo dalla copia del file su disco ne complica l'analisi forense post-infezione.

-----3. Dettagli dell'Accesso Iniziale (Initial Access Summary)

L'analisi del processo `firefox.exe` stabilisce il momento esatto dell'infiltrazione e la natura del file scaricato:-----

Evento Critico	Tattica MITRE ATT&CK	Rilevanza Tattica e Forense
Threat Verdict (94/100)	Accesso Iniziale	Indica che il browser, pur legittimo, è stato l'agente che ha introdotto il codice malevolo, un'anomalia comportamentale significativa.
Drops legitimate windows executable	T1036.003 (Masquerading)	Il file viene scritto in forma temporanea (<code>.part</code>) nella cartella Download . Questo segna il punto di transizione da traffico di rete a minaccia sul file system.
Reads Microsoft Office registry keys	T1012 (Discovery)	L'interrogazione delle chiavi di registro di Office può essere una tecnica di <i>Discovery</i> per determinare la presenza di software target (es. credenziali salvate) o un effetto collaterale dell'esecuzione del browser.

-----4. Inferenza Tattica: Motivazioni e Scopo dell'Architettura

L'architettura di attacco a doppia iniezione, abbinata all'evasione anti-sandbox e all'offuscamento, indica un malware mirato a obiettivi di alto valore con l'intento di **persistenza a lungo termine e massima elusione EDR**. Ipotesi 1: Ottimizzazione del Bypass EDR (*A/B Evasion Testing*)

L'attore della minaccia sta eseguendo una forma di *A/B Testing* in tempo reale per identificare la via di esecuzione più efficace contro la configurazione di sicurezza specifica della vittima:

- **Vettore LOLBin (installutil.exe)**: Molte soluzioni EDR possono rilevare l'iniezione in `installutil.exe` come un'anomalia. Se il Vettore 1 fallisce, il Vettore 2 garantisce l'esecuzione.
- **Vettore Self-Injection (muadnrd.exe)**: Questa tecnica può aggirare le regole EDR che si concentrano specificamente sul monitoraggio dell'attività sospetta di LOLBins, operando in uno spazio di memoria meno monitorato.

Ipotesi 2: Diversificazione Funzionale del Carico Utile (*Payload Splitting*)

La biforcazione potrebbe non essere solo evasiva, ma funzionale, indicando che i due rami caricano moduli di attacco diversi:

- **Modulo in LOLBin (Elevata Legittimità)**: Potrebbe ospitare il modulo più critico o rumoroso, come un **Information Stealer** o un modulo di esfiltrazione dati, beneficiando della reputazione del processo genitore.
- **Modulo Auto-Iniettato (Failsafe e Persistenza)**: Potrebbe essere un **Remote Access Trojan (RAT)** o un modulo di *beaconing* per il **Comando e Controllo (C2)**, configurato per garantire la persistenza e le comunicazioni silenziose a lungo termine.

Ipotesi 3: Robustezza e Tolleranza ai Guasti (Failsafe Mechanism)

L'architettura assicura l'esecuzione anche in scenari di errore:

- La presenza del nodo `werfault.exe` (Windows Error Reporting) suggerisce che il malware è progettato per gestire o addirittura indurre crash controllati, recuperando l'esecuzione dal ramo parallelo.
- Se l'iniezione in un processo .NET (come `installutil.exe`) dipende dall'integrità del *runtime* del framework, la **Self-Injection** offre una via d'esecuzione meno dipendente da componenti di sistema esterni.

In sintesi, il campione analizzato mostra un alto livello di sofisticazione, concentrandosi sull'elusione del rilevamento sia a riposo (tramite *packer* e traffico LOTS) sia in esecuzione (tramite *Process Hollowing* e *Self-Injection*).